



Sunbeam Institute of Information Technology
Pune and Karad

Data structures and Algorithms

Trainer - Devendra Dhande

Email – devendra.dhande@sunbeaminfo.com



Hashing

Array : linear search - $O(n)$
 binary search - $O(\log n)$
 Linked List : linear search - $O(n)$

} time will vary wot to input

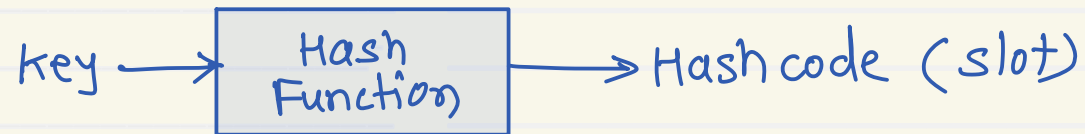
- hashing is a technique in which data can be inserted, deleted and searched in constant average time $O(1)$
- Implementation of hashing is known as hash table
- Hash table is array of fixed size in which elements are stored in key - value pairs

Array - Hash table

Index - Slot

↳ Associative data structure

- In hash table only unique keys are stored
- Every key is mapped with one slot of the table and this is done with the help of mathematical function known as hash function



key → value

88-V1
93-V2
100-V3
54-V4
76-V5

Size=10

100, V3	0
	1
	2
93, V2	3
54, V4	4
	5
76, V5	6
	7
88, V1	8
	9

Hash Table

$$h(k) = k \% \text{size}$$

hash function

$$h(88) = 88 \% 10 = 8 \leftarrow \text{hash code}$$

$$h(93) = 93 \% 10 = 3$$

$$h(100) = 100 \% 10 = 0$$

$$h(54) = 54 \% 10 = 4$$

$$h(76) = 76 \% 10 = 6$$

Add

1. Find slot = $h(k)$
2. $arr[slot] = \text{data}$

Search

1. Find slot = $h(k)$
2. return $arr[slot].value$

Delete

1. Find slot = $h(k)$
2. $arr[slot] = \text{null}$

constant time

$$T(n) = O(1)$$

Size = 10

key → value

88-V1
93-V2
100-V3
54-V4
76-V5
113-V6

Ⓢ

100, V3	0
	1
	2
93, V2	3
54, V4	4
	5
76, V5	6
	7
88, V1	8
	9

Hash Table

$$h(k) = k \% \text{ size}$$

$$h(88) = 88 \% 10 = 8$$

$$h(93) = 93 \% 10 = 3$$

$$h(100) = 100 \% 10 = 0$$

$$h(54) = 54 \% 10 = 4$$

$$h(76) = 76 \% 10 = 6$$

$$h(113) = 113 \% 10 = 3$$

Collision:

When two or more distinct keys yield/give same slot, called as collision.

Collision handling / resolution techniques:-

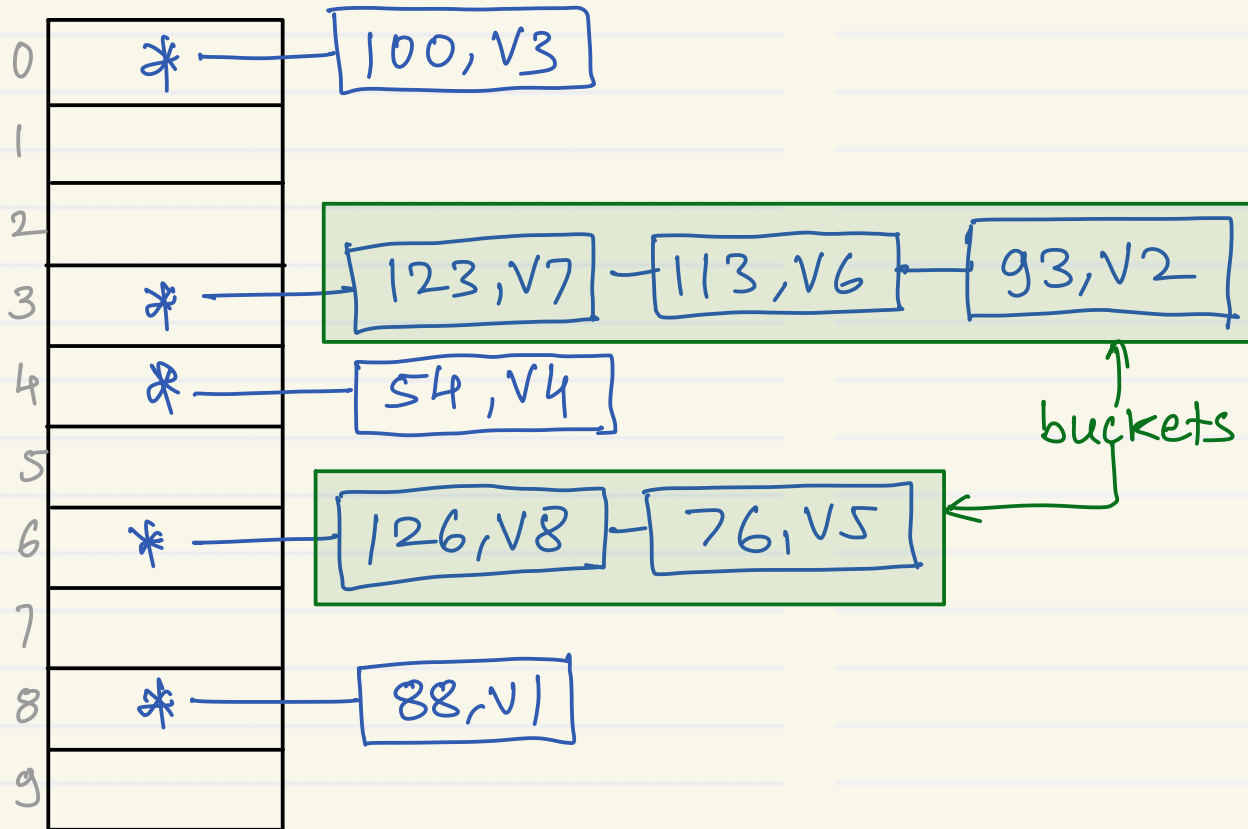
1. Closed Addressing (chaining)
2. Open Addressing
 - i. linear probing
 - ii. quadratic probing
 - iii. double hashing.

Closed Addressing / Chaining / Separate Chaining

Size = 10 (per slot one linked list is maintained)

$$h(k) = k \% \text{ size}$$

88-V1
93-V2
100-V3
54-V4
76-V5
113-V6
123-V7
126-V8



Hash Table

Advantage :
can store multiple key value pairs

Disadvantages:

- i. key value pairs are stored outside the table
- ii. space requirement is more
- iii. worst case time complexity is $O(n)$.
(if maximum keys yield same slot)

Open addressing - Linear probing

Size = 10

88-V1
93-V2
100-V3
54-V4
76-V5
113-V6

100, V3	0
	1
	2
93, V2	3
54, V4	4
113, V6	5
76, V5	6
	7
88, V1	8
	9

Hash Table

Probing:
finding next of the table on which key-value pair can be kept when collision will occur

$$h(k) = k \% \text{ size}$$

$$h(k, i) = [h(k) + f(i)] \% \text{ size}$$

$$f(i) = i$$

probe numbers

where $i = 1, 2, 3, \dots$

$$h(113) = 113 \% 10 = 3 \text{ (c)}$$

$$1^{\text{st}} \text{ probe : } h(113, 1) = [3 + 1] \% 10 = 4 \text{ (c)}$$

$$2^{\text{nd}} \text{ probe : } h(113, 2) = [3 + 2] \% 10 = 5$$

Primary clustering:

- near key position hash table becomes bulky / crowded.
- need to take long run of filled slots near key position to find empty slot

Open addressing - Quadratic probing

size = 10

88-V1
93-V2
100-V3
54-V4
76-V5
113-V6

100, V3	0
	1
	2
93, V2	3
54, V4	4
	5
76, V5	6
113, V6	7
88, V1	8
	9

Hash Table

$$h(k) = k \% \text{size}$$

$$h(k, i) = [h(k) + f(i)] \% \text{size}$$

$$f(i) = i^2$$

where $i = 1, 2, 3, \dots$

$$h(113) = 113 \% 10 = 3 \text{ (c)}$$

$$1^{\text{st}} \text{ probe: } h(113, 1) = [3 + 1] \% 10 = 4 \text{ (c)}$$

$$2^{\text{nd}} \text{ probe: } h(113, 2) = [3 + 4] \% 10 = 7$$

- primary clustering will be solved
- there is no guarantee of getting free slot

Secondary clustering:

- need to take long run of filled slots away key position to find empty slot

Open addressing - Quadratic probing

Size = 10

88-V1
93-V2
100-V3
54-V4
76-V5
113-V6
23-V7
33-V8

100, V3	0
	1
23, V7	2
93, V2	3
54, V4	4
	5
76, V5	6
113, V6	7
88, V1	8
33, V8	9

Hash Table

$$h(k) = k \% \text{ size}$$

$$h(k, i) = [h(k) + f(i)] \% \text{ size}$$

$$f(i) = i^2$$

where $i = 1, 2, 3, \dots$

$$h(23) = 23 \% 10 = 3 \text{ (C)}$$

$$h(23, 1) = [3 + 1] \% 10 = 4 \text{ (C)}$$

$$h(23, 2) = [3 + 4] \% 10 = 7 \text{ (C)}$$

$$h(23, 3) = [3 + 9] \% 10 = 2$$

$$h(33) = 33 \% 10 = 3 \text{ (C)}$$

$$h(33, 1) = [3 + 1] \% 10 = 4 \text{ (C)}$$

$$h(33, 2) = [3 + 4] \% 10 = 7 \text{ (C)}$$

$$h(33, 3) = [3 + 9] \% 10 = 2 \text{ (C)}$$

$$h(33, 4) = [3 + 16] \% 10 = 9$$

Open addressing - Double hashing

size=11

85 - V1

69 - V2

98 - V3

25 - V4

36 - V5

	0
	1
	2
69, V2	3
	4
	5
25, V4	6
	7
85, V1	8
36, V5	9
98, V3	10

Hash Table

$$h1(k) = k \% \text{size}$$

$$h2(k) = 7 - (k \% 7)$$

$$h(k, i) = [h1(k) + i * h2(k)] \% \text{size}$$

$$h1(85) = 85 \% 11 = 8$$

$$h1(69) = 69 \% 11 = 3$$

$$h1(98) = 98 \% 11 = 10$$

$$h1(25) = 25 \% 11 = 3 \text{ (c)}$$

$$h2(25) = 7 - (25 \% 7) = 3$$

$$h(25, 1) = [3 + 1 * 3] \% 11 = 6 \text{ (1st probe)}$$

$$h1(36) = 36 \% 11 = 3 \text{ (c)}$$

$$h2(36) = 7 - (36 \% 7) = 6$$

$$h(36, 1) = [3 + 1 * 6] \% 11 = 9$$

$$\text{Load factor} = \frac{n}{N}$$

(λ)

n - number of elements (key-value) present in hash table

N - number of total slots in hash table

$$n = 6, N = 10$$

$$\lambda = \frac{n}{N} = \frac{6}{10} = \underline{\underline{0.6}}$$

↓
Hash table is 60% filled

- Load factor ranges from 0 to 1.
- If $n < N$ Load factor < 1 - free slots are available
- If $n = N$ Load factor $= 1$ - free slots are not available
- if $n > N$ Load factor > 1

- not possible in case of open addressing

- In rehashing, whenever hash table will be filled more than 60 or 70 % size of hash table is increased by twice
- Existing key value pairs are remapped according to new size

open addr : threshold : 0.6 to 0.7

closed addr : threshold : 0.75 to 0.9

Two sum

Given an array of integers nums and an integer target, return indices of the two numbers such that they add up to target.

You may assume that each input would have exactly one solution, and you may not use the same element twice.

You can return the answer in any order.

Example 1:

Input: nums = [2,7,11,15], target = 9

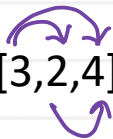
Output: [0,1]



Example 2:

Input: nums = [3,2,4], target = 6

Output: [1,2]



Example 3:

Input: nums = [3,3], target = 6

Output: [0,1]

$$T(n) = O(n^2)$$
$$S(n) = O(1)$$

```
int[] twoSum(int[] nums, int target) {  
    for(int i=0; i < nums.length; i++){  
        for(int j=i+1; j < nums.length; j++){  
            if(nums[i] + nums[j] == target){  
                return new int[] {i, j};  
            }  
        }  
    }  
    return new int[] { };  
}
```

Two sum

Given an array of integers nums and an integer target, return indices of the two numbers such that they add up to target.

You may assume that each input would have exactly one solution, and you may not use the same element twice.

You can return the answer in any order.

Example 1:

Input: nums = [2,7,11,15], target = 9

Output: [0,1]

HashMap

key	value
2	0

de → key ← index

Example 2:

Input: nums = [3,2,4], target = 6

Output: [1,2]

HashMap

key	value
3	0
2	1

Example 3:

Input: nums = [3,3], target = 6

Output: [0,1]

$$T(n) = O(n)$$

$$S(n) = O(n)$$

```
int[] twoSum(int[] nums, int target) {
    Map<Integer,Integer> tbl = new HashMap<>();
    for(int i=0; i< nums.length; i++){
        int diff = target - nums[i];
        if(tbl.containsKey(diff))
            return new int[] {tbl.get(diff), i};
        tbl.put(nums[i], i);
    }
    return new int[] { 3 };
}
```

Two Sum II - Input Array Is Sorted

Ascending

- Given a 1-indexed array of integers numbers that is already sorted in non-decreasing order, find two numbers such that they add up to a specific target number. Let these two numbers be numbers[index1] and numbers[index2] where $1 \leq \text{index1} < \text{index2} \leq \text{numbers.length}$.
- Return the indices of the two numbers index1 and index2, each incremented by one, as an integer array [index1, index2] of length 2.
- The tests are generated such that there is exactly one solution. You may not use the same element twice.
- Your solution must use only constant extra space.

two pointers technique

- Example 1:

- Input: numbers = [2,7,11,15], target = 9

- Output: [1,2]
- | | 0 | 1 | 2 | 3 | left | right | sum | |
|--|---|---|---|---|------|-------|-----|----|
| | | | | | 0 | 3 | 17 | >9 |
| | | | | | 0 | 2 | 13 | >9 |
| | | | | | 0 | 1 | 9 | =9 |

- Example 2:

- Input: numbers = [2,3,4], target = 6

- Output: [1,3]
- | | 0 | 1 | 2 | left | right | sum | |
|--|---|---|---|------|-------|-----|----|
| | | | | 0 | 2 | 6 | =6 |

- Example 3:

- Input: numbers = [-1,0], target = -1

- Output: [1,2]

```
int[] twoSum(int[] nums, int target) {
    int left = 0, right = nums.length - 1;
    while (left < right) {
        int sum = nums[left] + nums[right];
        if (sum == target)
            return new int[] {left + 1, right + 1};
        if (sum > target)
            right--;
        else
            left++;
    }
    return null;
}
```

Container With Most Water

- You are given an integer array height of length n. There are n vertical lines drawn such that the two endpoints of the ith line are (i, 0) and (i, height[i]).
- Find two lines that together with the x-axis form a container, such that the container contains the most water.
- Return the maximum amount of water a container can store.
- Notice that you may not slant the container.

Example 1:

Input: height = [1,8,6,2,5,4,8,3,7]

Output: 49

Explanation: The above vertical lines are represented by array [1,8,6,2,5,4,8,3,7]. In this case, the max area of water (blue section) the container can contain is 49.



Example 2:

Input: height = [1,1]

Output: 1

left	right	height	width	area
0	8	1	8	8
1	8	7	7	49
1	7	3	6	18
1	6	8	5	40
,	,	,	,	,
,	,	,	,	,

maxArea = 49

```
int maxArea(int [] height) {  
    int left=0, right = height.length-1;  
    int maxArea=0;  
    while (left < right) {  
        int area = ( right-left ) * Integer.min(height[left], height[right]);  
                    width           height  
        if (area > maxArea)  
            maxArea = area;  
        if (height[left] < height[right])  
            left++;  
        else  
            right--;  
    }  
    return maxArea;  
}
```

Contains Duplicate

Given an integer array nums, return true if any value appears at least twice in the array, and return false if every element is distinct.

Example 1:

Input: $nums = [1, 2, 3, 1]$

Output: true

HashSet

1, 2, 3

Example 2:

Input: $nums = [1, 2, 3, 4]$

Output: false

HashSet

1, 2, 3, 4

Example 3:

Input: $nums = [1, 1, 1, 3, 3, 4, 3, 2, 4, 2]$

Output: true

$$T(n) = O(n)$$
$$S(n) = O(n)$$

```
boolean containsDuplicate(int[] nums) {  
    Set<Integer> s = new HashSet<>();  
    for (int n : nums) {  
        if (s.add(n) == false)  
            return true;  
    }  
    return false;  
}
```

3

Valid Anagram

- Given two strings s and t, return true if t is an anagram of s, and false otherwise.

- Example 1:

- Input: s = "anagram", t = "nagaram"
- Output: true

- Example 2:

- Input: s = "rat", t = "car"
- Output: false

```
boolean isAnagram(String s, String t) {  
    if(s.length() != t.length())  
        return false;  
  
    int[] counts = new int[26];  
    char[] sarr = s.toCharArray();  
    char[] tarr = t.toCharArray();  
    for(int i=0; i<sarr.length; i++)  
        ++counts[sarr[i]-'a'];  
    for(int i=0; i<tarr.length; i++){  
        int x = --counts[tarr[i]-'a'];  
        if(x<0)  
            return false;  
    }  
    return true;  
}
```

Range Sum Query - Immutable

- Given an integer array nums, handle multiple queries of the following type:
- Calculate the sum of the elements of nums between indices left and right inclusive where left <= right.
- Implement the NumArray class:
- NumArray(int[] nums) Initializes the object with the integer array nums.
- int sumRange(int left, int right) Returns the sum of the elements of nums between indices left and right inclusive (i.e. nums[left] + nums[left + 1] + ... + nums[right]).
- Example 1:
 - Input
 - ["NumArray", "sumRange", "sumRange", "sumRange"]
 - [[[-2, 0, 3, -5, 2, -1]], [0, 2], [2, 5], [0, 5]]
 - Output
 - [null, 1, -1, -3]

```
class NumArray {  
    int nums[];  
  
    public NumArray(int nums[]) {  
        this.nums = nums;  
    }  
  
    public int sumRange(left, right) {  
        int sum = 0;  
        for (i = left; i <= right; i++) {  
            sum += this.nums[i];  
        }  
        return sum;  
    }  
}
```

Range Sum Query - Immutable

- Given an integer array nums, handle multiple queries of the following type:
- Calculate the sum of the elements of nums between indices left and right inclusive where left <= right.
- Implement the NumArray class:
- NumArray(int[] nums) Initializes the object with the integer array nums.
- int sumRange(int left, int right) Returns the sum of the elements of nums between indices left and right inclusive (i.e. nums[left] + nums[left + 1] + ... + nums[right]).
- Example 1:
 - Input
 - ["NumArray", "sumRange", "sumRange", "sumRange"]
 - [[[-2, 0, 3, -5, 2, -1]], [0, 2], [2, 5], [0, 5]]
 - Output
 - [null, 1, -1, -3]

arr	-2	0	3	-5	2	-1
	0	1	2	3	4	5

prefixSum	-2	-2	1	-4	-2	-3
	0	1	2	3	4	5

$$\text{prefixSum}[i] = \sum_{j=0}^i \text{arr}[j]$$

$$[2, 5] = 2$$

$$[0, 5] = -3$$

$$[0, 1] = -2$$

$$\begin{aligned} [2, 5] &= [0, 5] - [0, 1] \\ &= -3 - -2 \\ &= -1 \end{aligned}$$

```
class NumArray {
```

```
    int[] prefixSum;
```

```
    public NumArray(int nums[]) {
```

```
        prefixSum = new int[nums.length];
```

```
        prefixSum[0] = nums[0];
```

```
        for (int i = 1; i < nums.length; i++)
```

```
            prefixSum[i] = prefixSum[i-1] + nums[i];
```

```
    }
```

```
    public int sumRange(left, right)
```

```
    {
```

```
        int sumLeft = left == 0 ? 0 : prefixSum[left-1];
```

```
        return prefixSum[right] - sumLeft;
```

```
    }
```



Maximum Average Subarray I

- You are given an integer array `nums` consisting of `n` elements, and an integer `k`.
- Find a contiguous subarray whose length is equal to `k` that has the maximum average value and return this value. Any answer with a calculation error less than 10^{-5} will be accepted.

- Example 1:

- Input: `nums = [1,12,-5,-6,50,3]`, `k = 4`

- Output: 12.75000

- Explanation: Maximum average is $(12 - 5 - 6 + 50) / 4 = 51 / 4 = 12.75$

$$\begin{aligned} 1: & 1 + 12 - 5 - 6 = 2 \\ 2: & 12 - 5 - 6 + 50 = 51 \leftarrow \\ 3: & -5 - 6 + 50 + 3 = 42 \end{aligned}$$

$$\begin{aligned} 0-3 \\ 1-4 \\ 2-5 \end{aligned}$$

- Example 2:

- Input: `nums = [5]`, `k = 1`

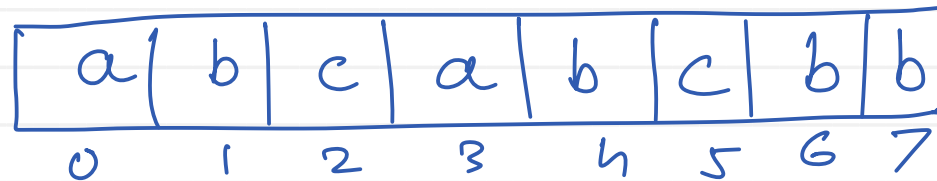
- Output: 5.00000

1	12	-5	-6	50	3
0	1	2	3	4	5

```
double findMaxAverage(int[] nums, int k) {  
    int sum = 0;  
    for(int i = 0; i < k; i++)  
        sum += nums[i];  
  
    int maxSum = sum;  
    for(int j = 0, i = k; i < nums.length; i++, j++) {  
        sum = sum - nums[j] + nums[i];  
        if (sum > maxSum)  
            maxSum = sum;  
    }  
    return maxSum / k;  
}
```

Longest Substring Without Repeating Characters

- Given a string s , find the length of the longest substring without duplicate characters.
- Example 1:
 - Input: $s = \text{"abcabcbb"}$
 - Output: 3
 - Explanation: The answer is "abc", with the length of 3. Note that "bca" and "cab" are also correct answers.
- Example 2:
 - Input: $s = \text{"bbbbbb"}$
 - Output: 1
 - Explanation: The answer is "b", with the length of 1.
- Example 3:
 - Input: $s = \text{"pwwkew"}$
 - Output: 3
 - Explanation: The answer is "wke", with the length of 3.
 - Notice that the answer must be a substring, "pwke" is a subsequence and not a substring.



```
int lengthOfLongestSubstring( String s) {  
    HashSet<Character> set = new HashSet<>();  
    int left = 0; maxlen = 0;  
    for( int right = 0; right < s.length(); right++) {  
        char ch = s.charAt(right);  
        while (set.contains(ch)) {  
            set.remove(s.charAt(left));  
            left++;  
        }  
        set.add(ch);  
        maxlen = Integer.max(maxlen, right - left + 1);  
    }  
    return maxlen;  
}
```